Building the Trade Funding Site with Payload

You already have the hard part done. The static site you built with Claude Code — the HTML, CSS and JavaScript — is your design. This guide turns it into a Payload site so you can edit content from a dashboard instead of touching code, then puts it live on a Vercel URL. Claude Code does the building; your job is to steer it and check the result. Custom domain and Cloudflare come at the very end, and only when you're ready.

5	~1 day	$0	Later
Steps start to live URL	Realistic first pass not counting content entry	To start free tiers cover it	DNS cutover optional final step

I. What you're actually building

The shift in one sentence

Right now the words on your site are baked into code. Payload pulls them out into a dashboard. Instead of asking a developer (or Claude) to change a headline, you'll log into an admin panel, edit the text, and hit save. The public site stays fast and secure because visitors never touch that dashboard — it's a separate door only your team has a key to.

This is the "headless CMS" idea from the first guide, made concrete. WordPress bolts the editor and the website together, and that seam is where the plugin security problems live. Payload keeps them apart.

The three pieces

You'll end up with three accounts, all free to start.

- **Payload** — the software that runs your site and the editing dashboard. It lives inside your project code; there's no separate signup.
- **Vercel** — where the site is hosted and served to the public. This is the same Vercel your team already used for the calculator site, so the account may exist.

- Neon — a free database that stores your content (the actual text and images). Vercel sets this up for you during deploy, so you rarely see it directly.

Cloudflare only enters the picture at the end, when you want `tradefunding.com.au` to point at the new site instead of the old one. Until then you work on a free `.vercel.app` web address.

One thing to know before you start

READ THIS

Payload needs to run inside a "Next.js" app, and your current site is plain HTML. That's fine — it just means the first real task is having Claude Code rebuild your existing pages as a Next.js app that looks identical. You are not redesigning anything. Same layout, same colours, same copy. You're changing the engine under the hood, not the bodywork.

Because Claude Code is doing the conversion, you don't need to understand Next.js. You need to describe what you have, ask for the conversion, and check that each page still looks right.

II. Set up your workspace (once)

Step 1 · Get the tools installed

You need three things on your computer: Node, a package manager, and Claude Code. If you built the existing site with Claude Code, Node and Claude Code are likely already there. To be sure, open your Terminal and check versions:

```
node --version    # needs to show v20.9 or higher
claude --version  # confirms Claude Code is installed
```

If `node --version` shows an older number or an error, install the current version from nodejs.org (choose the "LTS" button). Payload will not run on older Node.

Payload prefers a package manager called **pnpm**. Install it once with:

```
npm install -g pnpm
```

Step 2 · Open your existing site in Claude Code

Navigate Terminal to the folder that holds your current site, then launch Claude Code there. If your site lives in a folder called `trade-funding-site` on your Desktop:

```
cd ~/Desktop/trade-funding-site
claude
```

Claude Code now has your existing HTML, CSS and JavaScript in front of it. Everything below is a matter of telling it what you want in plain English. Copy the prompt boxes in this guide and paste them straight in.

First prompt — let Claude read your site

> *I have an existing static website built in plain HTML, CSS and JavaScript in this folder. Before we change anything, read through all the files and give me a short summary: what pages exist, what the main sections of each page are, and where the text and images live. Don't change any files yet.*

III. Convert the site to Payload

Step 3 · Give Claude one goal and the Payload docs

Instead of running the conversion and the CMS as two separate jobs, give Claude Code a single goal and point it at the official Payload documentation. Payload is a CMS platform that runs on Next.js, so both things — moving your site onto Next.js and putting Payload on top — are really one task. Handing Claude the docs matters: it makes sure it follows Payload's current, supported way of doing things instead of guessing.

The master prompt below tells Claude to do it in order: first convert the site to Next.js and install whatever it needs, then make sure the finished Next.js + Payload version matches your original design exactly. Paste it in, add the docs link, and let it work.

- Docs to reference: payloadcms.com/docs — the prompt already points Claude there.
- When it finishes it will tell you to run `pnpm dev` and open `localhost:3000`. Click through every page and compare it to your old site — flag any wrong font, missing image or broken link in plain words, and Claude will fix it.

The one master prompt — paste this whole thing

Try with Dia

> *I want to turn my existing static website into a site powered by Payload CMS. Payload is a CMS platform that runs on Next.js. Use the official Payload documentation as your reference for the correct, current way to do this: https://payloadcms.com/docs/getting-started/installation Do it in this order: 1. First, convert my existing HTML/CSS/JavaScript site into a Next.js application. Install whatever dependencies and set up whatever is needed for Payload to run on it (App Router, a supported Next.js version, and a database adapter — use SQLite locally so I don't have to set anything up). 2. Then make sure the converted Next.js + Payload version matches my original design exactly — same layout, fonts, colours, copy and images as the HTML/CSS/JavaScript I have now. Don't redesign anything. Set up these editable collections in Payload so my team can manage content from the dashboard: Pages (headline, body, images), Team Members (name, photo, title, bio), and Site Settings (phone, address, email, footer text). Wire the front-end pages to pull their content from Payload. When you're done, tell me exactly how to run it locally and how to log into the admin dashboard.*

Decide what's editable

YOUR CALL

The one decision that's genuinely yours to make: which parts of the site your team should be able to edit from the dashboard. The master prompt asks you to list these, so decide before you paste. For a Trade Funding marketing site, the usual pieces are:

- **Pages** — Home, About, each product page. Headline, body text, images.
- **Team members** — name, photo, title, bio (so adding a new hire doesn't need a developer).
- **Lender logos / partners** — if you show these anywhere.
- **Site settings** — phone number, address (Level 7, 233 Castlereagh St), email, footer text — the things that appear on every page.
- **News / blog posts** — only if you plan to publish articles.

Payload calls each of these a "collection." Anything you leave off stays fixed in code — which is fine, and actually safer.

About the database

DO NOTHING YET

Payload does require a database — there's no way around it — but you don't set one up now. Locally, Claude Code can point Payload at a simple file-based database (SQLite) so you can test everything on your own machine with zero setup. When you deploy to Vercel in the next section, Vercel creates a free Neon Postgres database for you and wires it up automatically. So the honest answer to "do I need to deal with a database?" is: not by hand, at any point in this guide.

Step 4 · Test the dashboard locally

Run the site on your own machine and log into the dashboard for the first time. With Claude Code done, run:

```
pnpm dev
```

Then open `localhost:3000/admin` in your browser. The first time, it asks you to create an admin account — that's your login for editing content. Create it, then try editing a headline, saving, and refreshing `localhost:3000` to see the change appear on the public page. If that loop works, the hard part is over.

> Green light to move on: you can change text in the dashboard and see it update on the site. If you can do that on your laptop, it will work the same way once it's live.

IV. Put it live on a Vercel URL

Step 5 · Push to GitHub, then deploy on Vercel

Vercel deploys from a GitHub repository, so the flow is: put the code on GitHub, connect it to Vercel, click deploy. Claude Code can handle the GitHub part for you — ask it to create a repository

and push the project. Then, in your browser:

- Go to vercel.com/new and sign in (use the team's existing account if there is one).
- Choose **Import** next to your new GitHub repository.
- Before deploying, open the **Storage** tab and add a **Neon Postgres** database and **Blob** storage (for images). Vercel injects the connection details automatically.
- Add one environment variable named `PAYLOAD_SECRET` — any long random string. Claude Code can generate one with `openssl rand -base64 32`.
- Click **Deploy**. A minute or two later you get a live web address ending in `.vercel.app`.

Visit `your-site.vercel.app/admin`, create your admin account again (the live database is separate from your laptop's), and you're editing the real, public site. **This is the finish line for now.** Share the `.vercel.app` link with Matt and Ben for review.

Prep the project for GitHub + Vercel

Try with Dia

> *Get this project ready to deploy on Vercel. Create a new private GitHub repository and push the code. Generate a secure PAYLOAD_SECRET value for me. Then give me a short, numbered checklist of exactly what to click in Vercel to deploy it, including adding a Neon Postgres database and Blob storage for images.*

A shortcut worth knowing

If the conversion in Section III turns out to be fiddly, there's a faster path. Payload publishes an official Vercel template that already includes the dashboard, database, image storage and a page builder — deployed with one click from the Payload Website Starter. You'd then point Claude at that template and have it move your existing design in, rather than converting from scratch — same master-prompt idea, different starting point. Worth mentioning to whoever reviews this; either route lands in the same place.

V. Later: the custom domain (optional)

When you're ready to use tradefunding.com.au

NOT YET

Only do this once the team has reviewed the Vercel URL and signed off. Pointing the real domain at the new site is a small task, but it's the one visitors notice, so it goes last. The domain's DNS is managed in Cloudflare, and the change is two entries.

- In Vercel, open your project → **Settings** → **Domains** and add `tradefunding.com.au`. Vercel shows you the exact records to create.

- In Cloudflare, open the `tradefunding.com.au` DNS settings and add the records Vercel gave you (usually an `A` record for the root and a `CNAME` for `www`).
- Set those records' proxy status to "DNS only" (grey cloud, not orange) so Vercel handles the security certificate cleanly.

Keep the current WordPress site untouched until the new one is approved. Because this is a domain switch that affects live customers, don't rush it — and loop in whoever manages the Cloudflare account so nothing else on the domain breaks.

When it's time — ask Claude to walk you through DNS

Try with Dia

> *The site is live on Vercel and approved. I now want to point our real domain tradefunding.com.au at it. Our DNS is managed in Cloudflare. Walk me through it one step at a time: what to do in Vercel first, then exactly which DNS records to add in Cloudflare and what proxy setting to use. Ask me to confirm after each step before moving on.*

VI. If something goes wrong

The universal fix

Paste the error into Claude Code and ask it to fix it. That sounds glib, but it's genuinely the right move nine times out of ten. Claude can see your whole project and the error together. Copy the full red text from the terminal or the browser, paste it in, and say "I got this error, please diagnose and fix it." Don't try to interpret it yourself.

Three specific gotchas

- **"Node version" errors on deploy** — Vercel is using an old Node. In Vercel project settings, set the Node version to 20.x or higher, then redeploy.
- **Images work locally but not when live** — you skipped Blob storage during deploy. Add it in Vercel's Storage tab and redeploy. Local uses your hard drive; live needs cloud storage.
- **Dashboard is empty after going live** — expected. The live database starts blank and is separate from your laptop. Re-enter (or ask Claude to help you export/import) your content once.

What "done" looks like

You're finished with this guide when three things are true: the site is live on a `.vercel.app` address, you can log into `/admin` and change content that shows up publicly, and Matt and Ben have the link to review. The custom domain is a separate, later decision — don't let it hold up sharing the working site.